Network Intrusion Detection System

Logistic Regression Analysis Report

Document Overview: This document outlines the methodology, data generation process, model training, and evaluation results for automating the detection of network intrusions. It details the use of Logistic Regression to classify network traffic as normal or suspicious based on key indicators such as packet count, byte transfer volume, and connection frequency.

Author: Nakedi Tumelo Peta

Date: January 16, 2026

Contents

1. Project Overview 3
2. Data Specification 3
2.1 Dataset Structure 3
2.2 Feature Dictionary 3
3. Exploratory Data Analysis (EDA) 4
4. Methodology 5
4.1 Data Preprocessing 5
4.2 Model Selection 5
5. Model Evaluation & Results 5
5.1 Quantitative Metrics 5
5.2 Confusion Matrix Analysis 5
5.3 ROC Curve Analysis 6
5.4 Feature Importance (Coefficients) 6
6. Conclusion 7
Appendix A: Python Source Code 7

Network Intrusion Detection System (NIDS)

1. Project Overview

This project implements a **Logistic Regression** model to monitor and classify network traffic patterns. By analyzing traffic metadata (such as packet volume and connection frequency), the system identifies potential network intrusions or anomalies.

Objective: To distinguish between **Normal** traffic and **Suspicious** traffic (e.g., potential DDoS or port scanning activity) with high probability.

2. Data Specification

The model analyzes a synthetic dataset representing network connection logs.

2.1 Dataset Structure

- **Sample Size (N):** 2,000 observations.
- **Class Balance:**
    - Normal Traffic (0): 1,685 samples (84.2%)
    - Suspicious Traffic (1): 315 samples (15.8%)

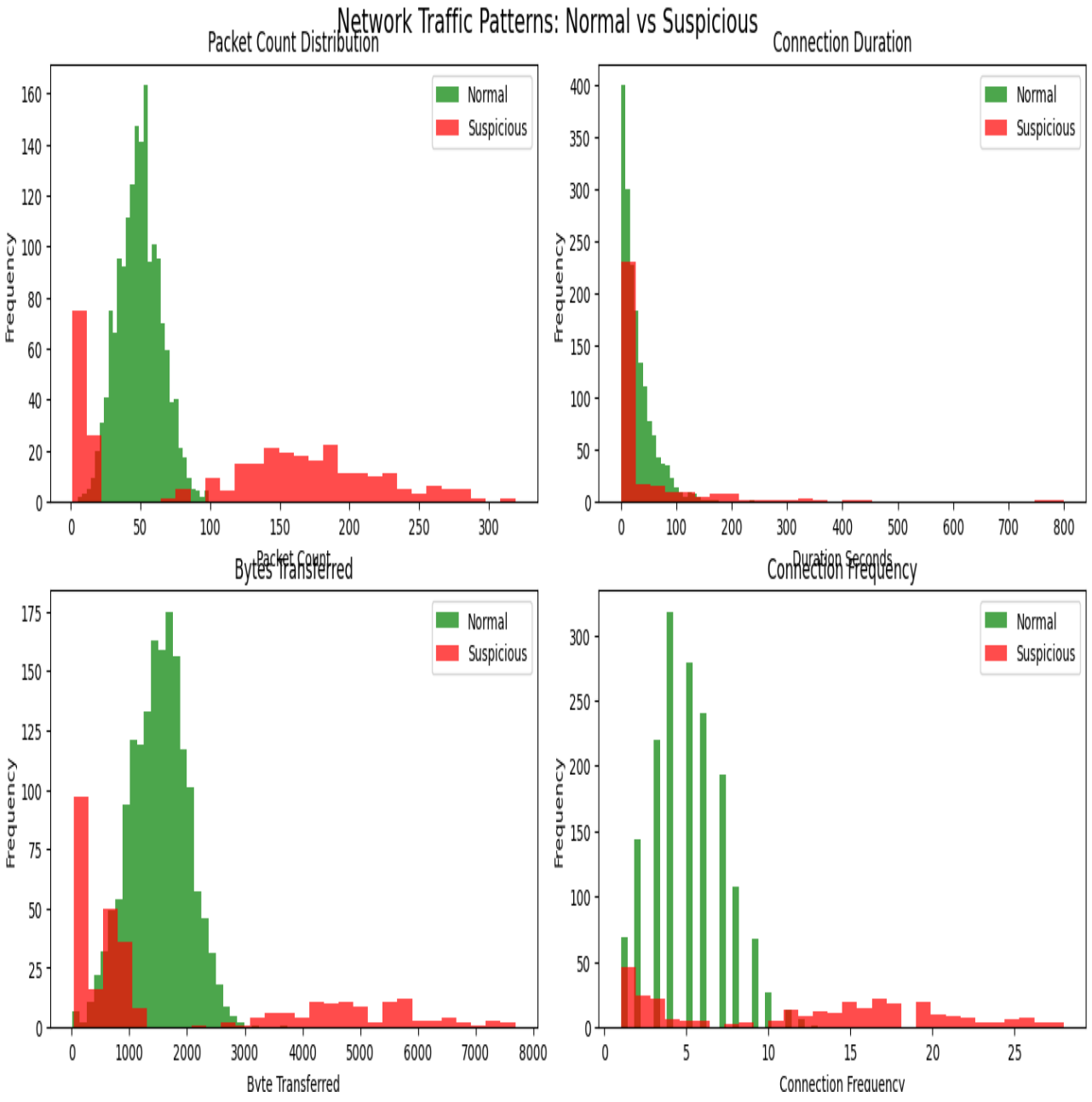
2.2 Feature Dictionary

Feature Name	Type	Description
packet_count	Integer	Total number of packets transferred during the session.
duration_seconds	Float	The duration of the connection session.
byte_transferred	Integer	Total data volume transferred (in bytes).
connection_frequency	Integer	Number of connection attempts in a short time window.
is_suspicious	Binary	**Target Variable.** (0 = Normal, 1 = Suspicious).

3. Exploratory Data Analysis (EDA)

Visual analysis reveals distinct patterns distinguishing normal from suspicious activity.

- **Packet Count & Frequency:** Suspicious traffic (shown in red) consistently exhibits higher packet counts and connection frequencies compared to normal traffic.
- **Distribution Overlap:** While some overlap exists in connection duration, the "Connection Frequency" feature shows a clear separation, making it a prime candidate for detection.



4. Methodology

4.1 Data Preprocessing

- **Standardization:** Because features had vastly different scales (e.g., duration ≈ 5.0 vs bytes ≈1600), a **StandardScaler** was applied. This ensures the Logistic Regression model weighs features correctly based on their importance, not their raw magnitude.

4.2 Model Selection

- **Algorithm:** Logistic Regression.
- **Reasoning:** Logistic Regression is ideal for binary classification where estimating the *probability* of an attack is required. It provides interpretable coefficients that rank risk factors.

5. Model Evaluation & Results

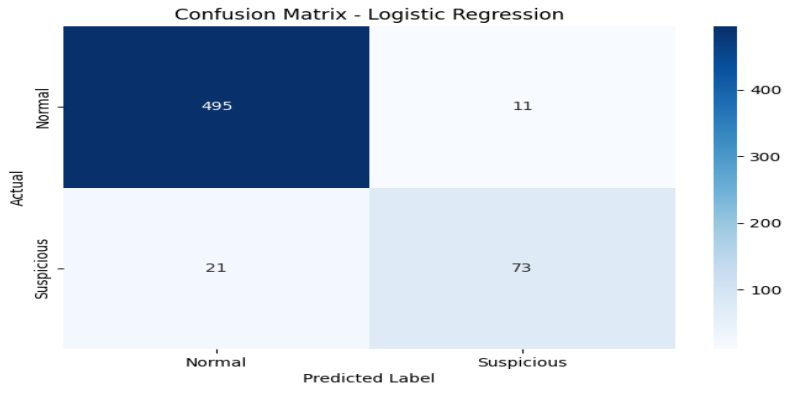
The model was evaluated on a test set of **600 samples**.

5.1 Quantitative Metrics

- **Accuracy: 94.7%** (Overall correctness).
- **Precision (Suspicious): 86.9%** (When the model flagged an alert, it was correct 87% of the time).
- **Recall (Suspicious): 77.7%** (The model successfully detected 78% of all actual attacks).
- **AUC Score: 0.968** (Excellent discrimination capability).

5.2 Confusion Matrix Analysis

The confusion matrix (**Figure 2**) breaks down the predictions:

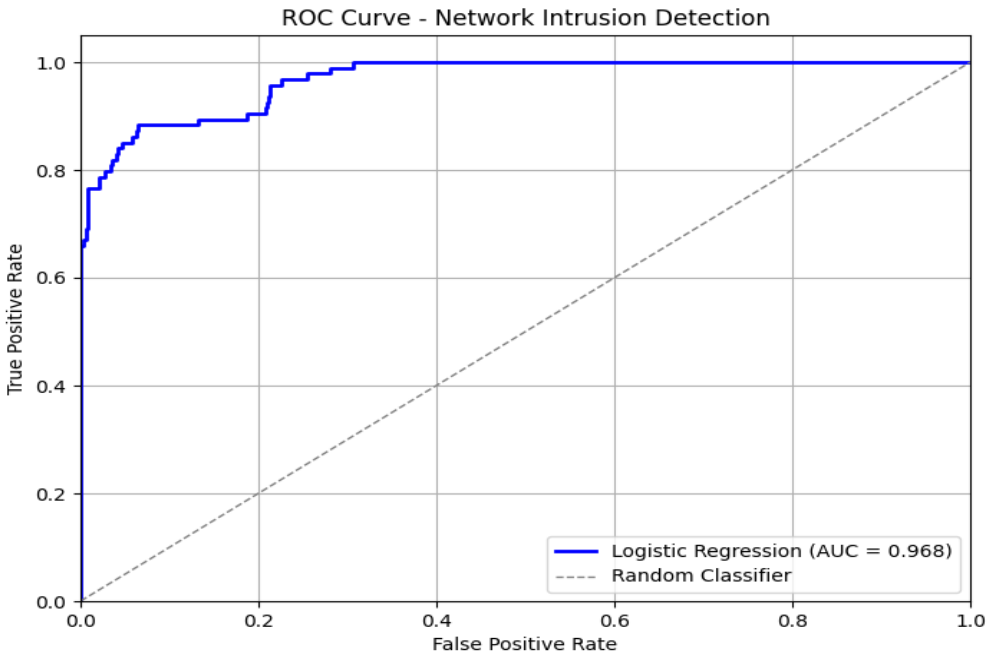


	Predicted Normal	Predicted Suspicious
Actual Normal	**495** (True Negative)	**11** (False Positive)
Actual Suspicious	**21** (False Negative)	**73** (True Positive)

- **False Positives (11):** A small number of legitimate connections were flagged as threats (Low false alarm rate).
- **False Negatives (21):** The model missed 21 suspicious events. Tuning the "Decision Threshold" could lower this number if security needs to be tighter.

5.3 ROC Curve Analysis

The ROC Curve (**Figure 3**) shows the trade-off between sensitivity and false alarms. The curve (blue line) rises steeply toward the top-left, confirming the high AUC of 0.968.



5.4 Feature Importance (Coefficients)

The Logistic Regression coefficients reveal which behaviours are most "suspicious":

1. **Connection Frequency (+3.374):** The strongest indicator. Rapid, repeated connections are highly correlated with attacks.
2. **Packet Count (+1.462):** High volumes of packets also indicate suspicious activity.
3. **Duration (+0.223):** Has the least impact on the classification.

6. Conclusion

The Logistic Regression model serves as an effective Network Intrusion Detection System (NIDS). With nearly 95% accuracy and a strong ability to identify high-frequency connection attacks, it provides a reliable baseline for automated network monitoring.

Next Steps

- **Threshold Tuning:** Adjust the probability threshold (currently 0.5) to reduce False Negatives (missed attacks) if the priority is catching *every* threat.
- **Deployment:** Integrate the model into a live packet capture stream.

Appendix A: Python Source Code

Description:

The script below generates the synthetic traffic logs, standardizes the data, trains the Logistic Regression model, and outputs the performance metrics used in this report.

Code Listing:

(network_detection.py)

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


# Scikit-learn imports

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression  # Fixed capitalization

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve

from sklearn.ensemble import VotingClassifier, RandomForestClassifier

import warnings

warnings.filterwarnings('ignore')
```

```
#Set plotting style
plt.style.use('default')
sns.set_palette("husl")

#-------------------------------------------------------------------------------------------
#Generate Realistic Network Traffic Data
#-------------------------------------------------------------------------------------------

#Create synthetic network traffic data that mimics real-world scenarios

np.random.seed(42) #For reproductive results

def generate_network_data(n_samples=10000):
    """
    Generate synthetic network traffic data with realistic patterns.
    """
    data = []

    for _ in range(n_samples):
        # Randomly decide if this is normal (0) or suspicious (1) traffic
        is_suspicious = np.random.choice([0, 1], p=[0.85, 0.15]) #15% suspicious

        if is_suspicious == 0: #Normal traffic
            packet_count = np.random.normal(50, 15) #Moderate packet count
            duration = np.random.exponential(30) #Short duration
            byte_sent = np.random.normal(1500, 500) #Moderate data size
            connection_freq = np.random.poisson(5) #Low frequency
```

```
        else: #Suspicious traffic
            # Suspicious traffic patterns: either very high or very low values
            attack_type = np.random.choice(['DDoS', 'Port Scan', 'Data Exfiltration'])

            if attack_type == 'DDoS':
                packet_count = np.random.normal(200, 50) #Very high packet count
                duration = np.random.exponential(5) #Very short duration
                byte_sent = np.random.normal(800, 200) #Very high data size
                connection_freq = np.random.poisson(20) #Very high frequency

            elif attack_type == 'Port Scan':
                packet_count = np.random.normal(10, 5) #Very few packet count
                duration = np.random.exponential(2) #Very short duration
                byte_sent = np.random.normal(200, 50) #Tiny packets
                connection_freq = np.random.poisson(15) #High frequency

            elif attack_type == 'Data Exfiltration':
                packet_count = np.random.normal(150, 30) #Moderate packet count
                duration = np.random.exponential(120) #Longer duration
                byte_sent = np.random.normal(5000, 1000) #Large data size
                connection_freq = np.random.poisson(2) #Low frequency

    # Ensure positive values
        packet_count = max(1, packet_count)
        duration = max(0.1, duration)
        byte_sent = max(10, byte_sent)
        connection_freq = max(1, connection_freq)

        data.append({
```

```
            'packet_count': int(packet_count),
            'duration_seconds': round(duration, 2),
            'byte_transferred': int(byte_sent),
            'connection_frequency': int(connection_freq),
            'is_suspicious': is_suspicious
        })

    return pd.DataFrame(data)

#Generate the dataset
df = generate_network_data(2000)
print("Dataset created!")
print(f"Total samples: {len(df)}")
print(f"Normal traffic: {len(df[df['is_suspicious'] == 0])} ({len(df[df['is_suspicious'] == 0]) / len(df) * 100:.1f}%)")
print(f"Suspicious traffic: {len(df[df['is_suspicious'] == 1])} ({len(df[df['is_suspicious'] == 1]) / len(df) * 100:.1f}%)")
print("\nFirst few samples:")
print(df.head())

#-----------------------------------------------------------------------------------------
#EDA (Exploratory Data Analysis)
#-----------------------------------------------------------------------------------------

fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('Network Traffic Patterns: Normal vs Suspicious', fontsize=16)

features = ['packet_count', 'duration_seconds', 'byte_transferred', 'connection_frequency']
```

```
title = ['Packet Count Distribution', 'Connection Duration', 'Bytes Transferred',
'Connection Frequency']


for i, (feature, title) in enumerate(zip(features, title)):
    row, col = i // 2, i % 2


    # Create histograms for normal and suspicious traffic
    normal_data = df[df['is_suspicious'] == 0][feature]
    suspicious_data = df[df['is_suspicious'] == 1][feature]


    axes[row, col].hist(normal_data, alpha=0.7, label='Normal', bins=30, color='green')
    axes[row, col].hist(suspicious_data, alpha=0.7, label='Suspicious', bins=30,
color='red')
    axes[row, col].set_title(title)
    axes[row, col].set_xlabel(feature.replace('_', ' ').title())
    axes[row, col].set_ylabel('Frequency')
    axes[row, col].legend()


plt.tight_layout()
plt.show()


#Display summery statistics
print("\nSummary Statistics by traffic type:")
print(df.groupby('is_suspicious')[features].describe().round(2))


#-----------------------------------------------------------------------------------------
#Data Preprocessing
#-----------------------------------------------------------------------------------------


#Separate features and target variable
```

```
X = df [['packet_count', 'duration_seconds', 'byte_transferred',
'connection_frequency']]

y = df['is_suspicious']


print("Feature matrix shape: ", X.shape)

print("Target vector shape: ", y.shape)


#Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)


print(f"Training set: {X_train.shape[0]} samples")

print(f"Testing set: {X_test.shape[0]} samples")


#Scale the features (important for logistic regression)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


print("\nFeatures have been standardized!")

print("Training features mean:", np.round(X_train_scaled.mean(axis=0), 3))

print("Training features std:", np.round(X_train_scaled.std(axis=0), 3))


#-------------------------------------------------------------------------------------------

#Build and train logistic regression model

#-------------------------------------------------------------------------------------------

#Create and train the logistic regression model

lr_model = LogisticRegression(random_state=42)

lr_model.fit(X_train_scaled, y_train)
```

```
print("\nLogistic Regression model trained!")


#Make predictions
y_pred_lr = lr_model.predict(X_test_scaled)
y_pred_proba_lr = lr_model.predict_proba(X_test_scaled)[:, 1] #Probability of being suspicious


print(f"Predictions made on {len(X_test)} test samples.")


#Display model coefficients
feature_names = ['packet_count', 'duration_seconds', 'byte_transferred', 'connection_frequency']
coefficients = lr_model.coef_[0]


print("\nModel Interpretable - Feature Coefficients:")
for feature, coef in zip(feature_names, coefficients):
    direction = "increases" if coef > 0 else "decreases"
    print(f"{feature}: {coef:.3f} - {direction} suspicious probability")



#-----------------------------------------------------------------------------------------
#Evaluate Model Peerformance
#-----------------------------------------------------------------------------------------


#Calculate performance metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


accuracy = accuracy_score(y_test, y_pred_lr)
precision = precision_score(y_test, y_pred_lr)
recall = recall_score(y_test, y_pred_lr)
```

```
f1 = f1_score(y_test, y_pred_lr)
auc_score = roc_auc_score(y_test, y_pred_proba_lr)

print("Logistic Regression Model Performance:")
print(f"Accuracy: {accuracy:.3f} - Overall correct predictions")
print(f"Precision: {precision:.3f} - Of flagged connections, how many were truly suspicious")
print(f"Recall: {recall:.3f} - Of all suspicious connections, how many were correctly identified")
print(f"F1 Score: {f1:.3f} - Balance between precision and recall")
print(f"AUC Score: {auc_score:.3f} - Model's ability to distinguish between classes")

#Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr, target_names=['Normal', 'Suspicious']))

#Confusion Matrix
plt.figure(figsize=(8, 5))
cm = confusion_matrix(y_test, y_pred_lr)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Suspicious'],
            yticklabels=['Normal', 'Suspicious'])
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted Label')
plt.ylabel('Actual')
plt.show()

#ROC Curve
plt.figure(figsize=(8, 6))
fpr, tpr, _ = roc_curve(y_test, y_pred_proba_lr)
```

```
plt.plot(fpr, tpr, color='blue', lw=2, label=f'Logistic Regression (AUC = {auc_score:.3f})')
plt.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--', label='Random Classifier')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Network Intrusion Detection')
plt.legend()
plt.grid(True)
plt.show()
```